AI Engineer World's Fair

EMBEDDING GEOMETRY · LOOP · AGENTIC SEARCH

Autoresearch & test-time compute for information retrieval

SCAN FOR THE SLIDES

hanxiao.io/aie-sf-2026

Han Xiao
VP of AI, Elastic

@hxiao · in/hxiao87 · arXiv:2605.11374

TEST-TIME COMPUTE

Spend more compute at inference, get a better answer.

Trade inference compute for quality: rather than a bigger model, the model does more work per query, along a smooth cost-versus-quality curve.

BEST-OF-N	sample many candidates, keep the best.
SELF-CONSISTENCY	majority-vote over sampled chains of thought.
VERIFIER SEARCH	spend inference FLOPs, climb a Pareto curve.

> Having a bot think for just **20 seconds** in a hand of poker got the same performance boost as scaling up the model by **100,000×** and training it for 100,000 times longer.
>
> Noam Brown, OpenAI, 2024

THE REFRAME

Search is test-time compute.

You wire trained embeddings, rerankers, multi-vector retrievers, and query expanders into a **pipeline at inference** to squeeze out relevance. You do not need a bigger model. You assemble more search at test time.

DON'T SCALE IT

one keyword match. Cheap, and probably not good enough.

SCALE IT

embed, retrieve, rerank, expand, fuse. More inference can buy more relevance.

The question is not whether the model is big enough. It is how much search pipeline you assemble at test time, and whether it pays off.

TWO VERSIONS

Two ways to manufacture that pipeline at test time.

A RECOMBINE THE GEOMETRY

An agentic loop writes **programs** over one frozen encoder: chunk, z-score, fuse channels, feed back. The pipeline is multi-pass embedding algebra.

the paper · this talk's core

B COMPOSE THE TOOLS

A small agent LLM **wires retrieval tools** (grep, embed, rerank, select_diverse) over a corpus under a budget. The pipeline is an agentic tool chain.

searchbox · later

VERSION A

"Small models gain nothing." But they are distilled from LLMs.

Test-time compute is said to belong to large reasoning models, with small models having nothing to gain. Yet today's embedders - including the small deployable ones - are distilled or adapted from LLM backbones:

Mistral-7B →	E5-Mistral, SFR-Embedding, GritLM, NV-Embed, bge-en-icl
Qwen3 →	qwen3-embed, jina-embeddings-v5
Gemma 3 →	EmbeddingGemma
LLaMA →	RepLLaMA, LLM2Vec

The lineage from a large language model to a small deployable encoder is now the dominant recipe, not the exception.

If test-time compute lives in the LLM representation space, these distilled encoders should inherit it. Do they?

THE INTUITION

Scoring runs from one cosine to late interaction.

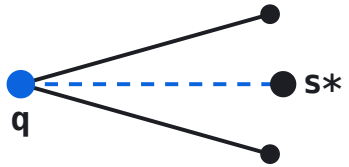
SINGLE-VECTOR COSINE

$\cos(q,\, d)$

one vector per document

FROZEN BASELINE

SENTENCE MAXSIM

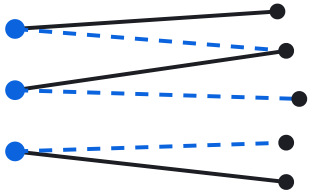


$\max_s \ \cos(q,\, s)$

same encoder, document split into sentence vectors

TEST-TIME STRUCTURE

COLBERT MULTI-VECTOR



$\sum_i \max_j \ \cos(q_i,\, d_j)$

every query token, max over all doc tokens

NEEDS A MULTI-VECTOR MODEL

THE QUESTION

How much can a frozen single-vector encoder gain at inference alone?

NO RETRAINING | NO AUXILIARY MODEL | NO LEARNED PARAMETERS | ONE FROZEN ENCODER API

The prominent recent test-time methods for retrieval each break at least one rule:

- **HYDE / QUERY2DOC** — an external LLM in the query path.
- **GQR** — a second, supervisory retriever.
- **METAEMBED** — trained extra parameters at deployment.

We forbid all three, and ask whether the gain scales with the compute spent.

AUTORESEARCH

Autoresearch: let the agent climb the hill.

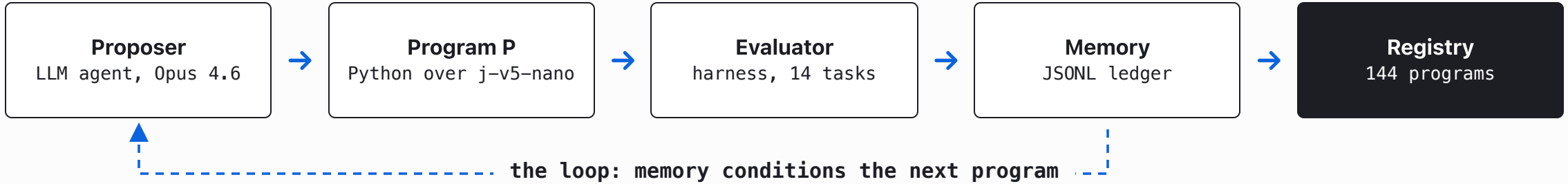
An agent runs the research loop itself: change one file, run a short fixed-budget experiment, keep the change if the metric improved, otherwise revert. Repeat, overnight. It is hill-climbing, with an LLM as the mutation function.

`change a file` → `run a short experiment` → `keep if better, else revert` ↻

> **You're not touching any of the Python files like you normally would as a researcher. Instead, you are programming the `program.md` files that set up your autonomous research org.**
>
> Andrej Karpathy, Anthropic, 2026

METHOD

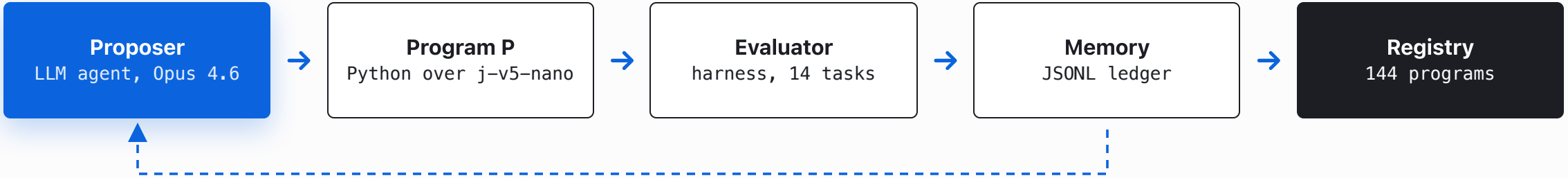
An LLM agent writes programs over the frozen encoder, and a harness scores them.



WRITES	THE ENCODER	SCORES ON	LOGS	AFTER G = 144
1 program per generation	239M, frozen small + fast	14 MMTEB tasks ΔnDCG@10	Δ, cost, parent, lesson per program	144 programs scored

METHOD · 1 PROPOSER

The proposer is an LLM, used as the mutation function.



DESIGN

Opus 4.6 reads the current frontier source and the memory ledger, edits **one** Python program, and proposes the next. Keep it if the metric improves, else revert. No human in the inner loop.

CAVEAT

It optimizes **exactly** the metric you hand it - not the one you mean. Reward the in-domain mean and reward spending compute, and that is what it will chase. Whether the gain survives out of domain is a separate question. The objective is the steering wheel.

METHOD · 2 PROGRAM

The program is arbitrary Python over the frozen encoder.

DESIGN

```
rerank(q_emb, d_emb, sim, *,
       embed_fn, q_texts, d_texts) -> scores
```

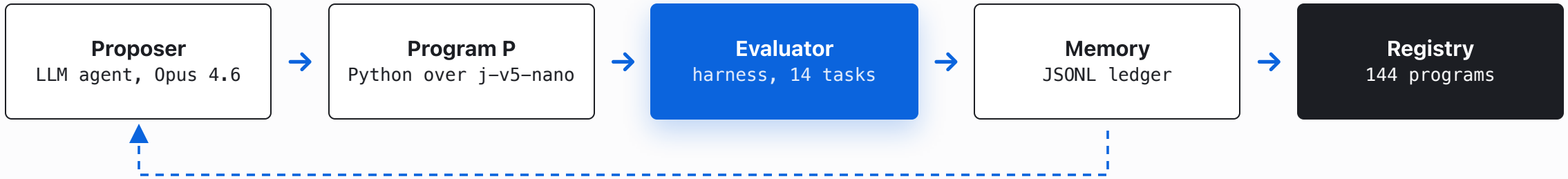
embed_fn is the test-time-compute budget. It re-embeds any text, switches the LoRA adapter (retrieval / passage / classification / matching), picks a Matryoshka dimension, or caps length. One call is one unit of compute.

CAVEAT

Auto-rejected: hyperparameters, stochastic ops, task-specific routing, external models, learned parameters, trivial constant-only variants. The taboos force **task-agnostic structure**, not a per-task tuned config.

METHOD · 3 EVALUATOR

The harness scores every program on the same 14 tasks.



DESIGN

Every program runs on the same **14 MMTEB Tier-1 discovery tasks** (legal, financial, long-document, general). The harness scores `ΔnDCG@10` against the cosine baseline, plus a cost ratio (`embed_fn` calls per query). The same fixed budget every generation.

CAVEAT

The loop only ever sees these 14 tasks. The **19 held-out tasks never enter the loop**, so a program can win in-domain and still fail out of domain. That gap is the entire experiment.

METHOD · 4 MEMORY

Every program is logged, and the log conditions the next one.

DESIGN

A JSONL ledger, one row per program: per-task `ΔnDCG@10`, win/tie/loss, cost ratio, parent, a novelty claim, and a post-hoc lesson. The proposer reads the frontier source plus this history, so each round builds on the last.

CAVEAT

Compounding cuts both ways: memory builds on real wins, but it also compounds whatever bias the objective has. A biased metric does not just mislead one program, it steers the whole lineage.

SETUP

What we search on, and what we test on.

DISCOVERY · THE LOOP SEARCHES HERE

j-v5-nano
239M · Jina
THE SEARCH ENCODER

small + fast → quick loop cycles

SEARCHED OVER

14 MMTEB Tier-1 tasks · legal, financial, long-doc, general

→

one program, must generalize

HELD-OUT · THE LOOP NEVER SEES THESE

j-v5-small
568M · Jina
SAME FAMILY

gemma-300m
303M · Gemma 3
UNSEEN FAMILY

qwen3-0.6b
600M · Qwen3
UNSEEN FAMILY

unseen families → the real transfer test

TESTED ON

19 MMTEB Tier-2/3 tasks · none in discovery (summarization, QA, fact-check, ...)

Metric: `ΔnDCG@10`, the program's nDCG@10 minus the cosine baseline. Gemma and Qwen share no training data, tokenizer, or adapter with the discovery model; qwen3-0.6b runs under a token-length cap.

THE DISTINCTION

Cost is one number: extra forward passes through the encoder.

$$c = 1 + \frac{\text{extra forward passes}}{\text{baseline passes}}$$

TEST-TIME STRUCTURE · C = 1

SoftCentroid: average the top-k document vectors you *already* computed, mix into the query, re-score.

```
q′ = q + mean(d_emb[top-k])
```

Zero extra forward passes - only algebra on vectors you already have.

TEST-TIME COMPUTE · C > 1

FirstSent: take the top doc, *re-embed* its first sentence, mix into the query, re-score.

```
q′ = q + embed_fn(first sentence)
```

One extra forward pass per query - new text through the encoder.

One reuses the geometry you already have; the other spends a forward pass on new text. Which converts into quality that transfers?

TWO SEARCHES, SAME LOOP

We run the search under two admission rules.

COMPUTE-SPENDING

Admit a program if its in-domain mean ΔnDCG@10 beats every prior one. The proposer is encouraged to spend more inference.

→ how high can the in-domain frontier climb?

TRANSFER-SELECTING

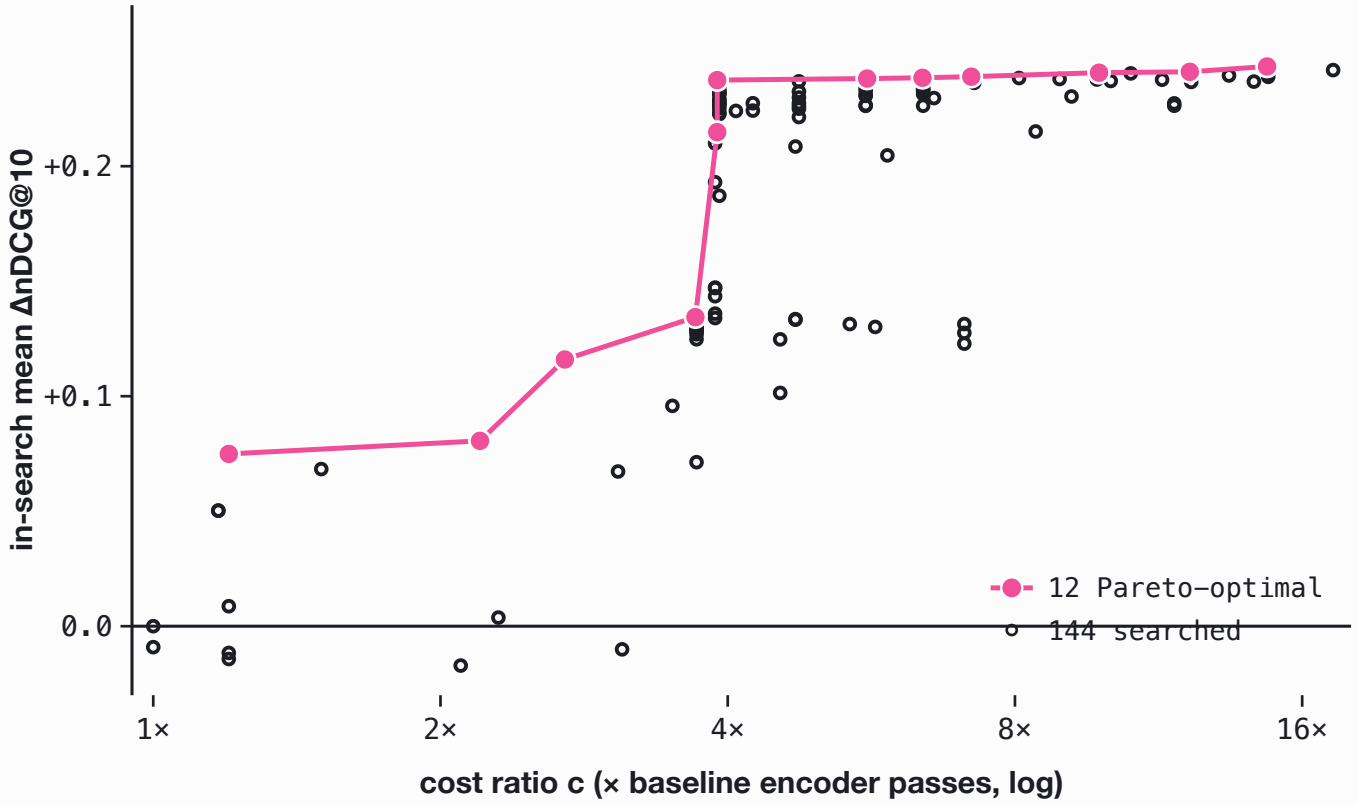
Admit only if a held-out validation split improves with no task regressing past 0.05. No reward for spending compute.

→ what holds up out of domain?

Neither search ever sees the 19 held-out evaluation tasks or the unseen encoder families.

WHAT COMPUTE BUYS, IN SEARCH

Told to spend compute, the search draws a clean Pareto curve.



144

programs searched over 144 generations

12

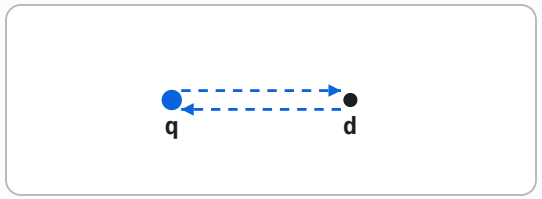
Pareto-optimal, from $c = 1.2$ to 14.7

In-search mean ΔnDCG@10 climbs from +0.07 to +0.24. This looks exactly like test-time-compute scaling.

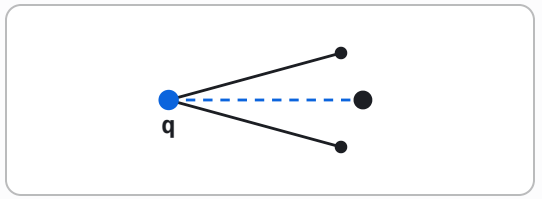
In-search only. The held-out test comes next.

WHAT THE SEARCH WROTE

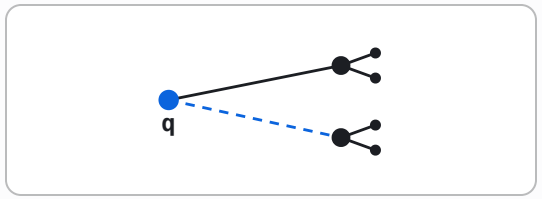
Twelve programs on the compute-spending frontier.



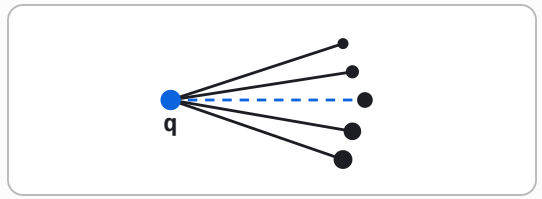
BidirZScore c=1.2



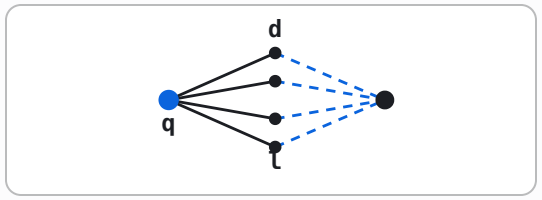
SentMaxSim c=2.2



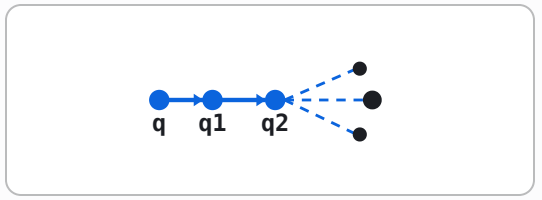
AdaptGranularity c=2.7



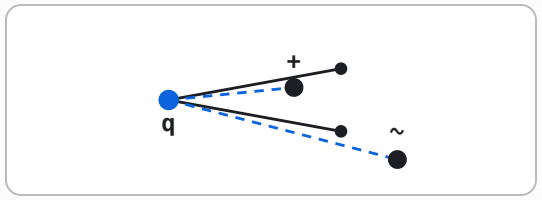
CoverageTriple c=3.7



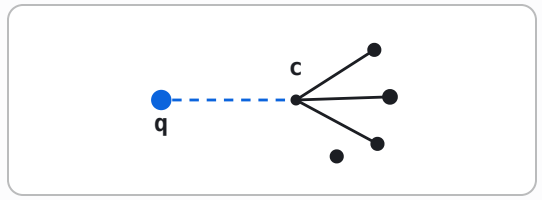
LexicalHybridRRF c=3.9



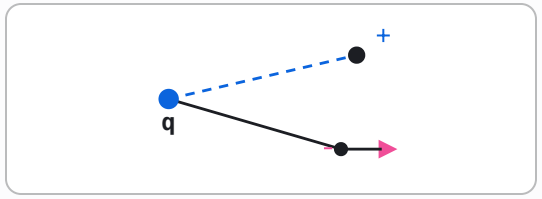
CrossRoundRRF c=3.9



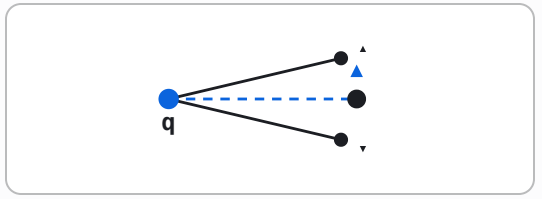
DiverseDualCtx c=5.6



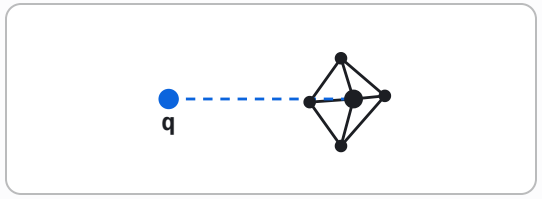
ConsensusRocchio c=6.4



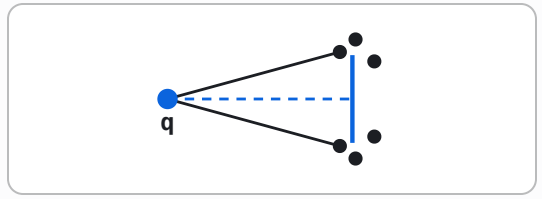
NegContrastive c=7.2



MomentumProg c=9.8



GraphCentrality c=12.2

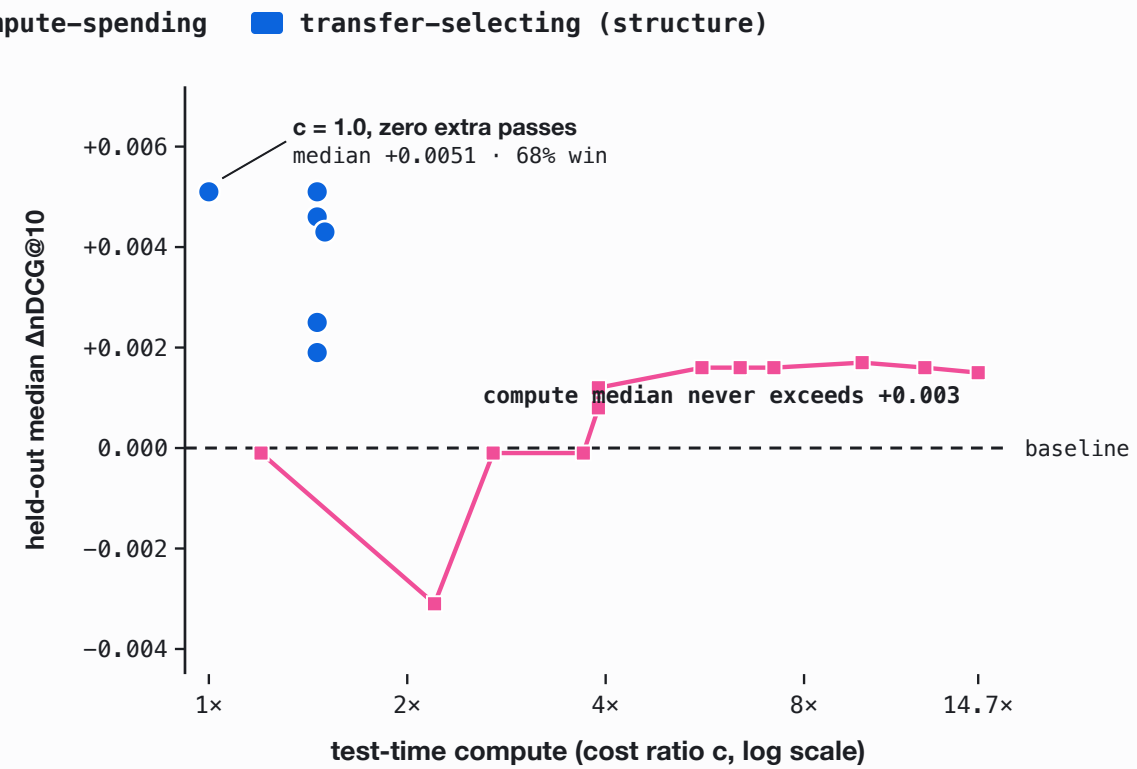


FisherStability c=14.7

Every one is a training-free recombination of the same frozen vectors. Cost climbs left to right; the gains, as the next slide shows, do not.

WHAT IT BUYS ON HELD-OUT DATA

On unseen encoders, compute is flat. Cheap structure is not.



14.7×

the most compute a program spends, beaten flat by a zero-pass program at $c = 1.0$

−0.016

compute's pooled held-out mean, below baseline

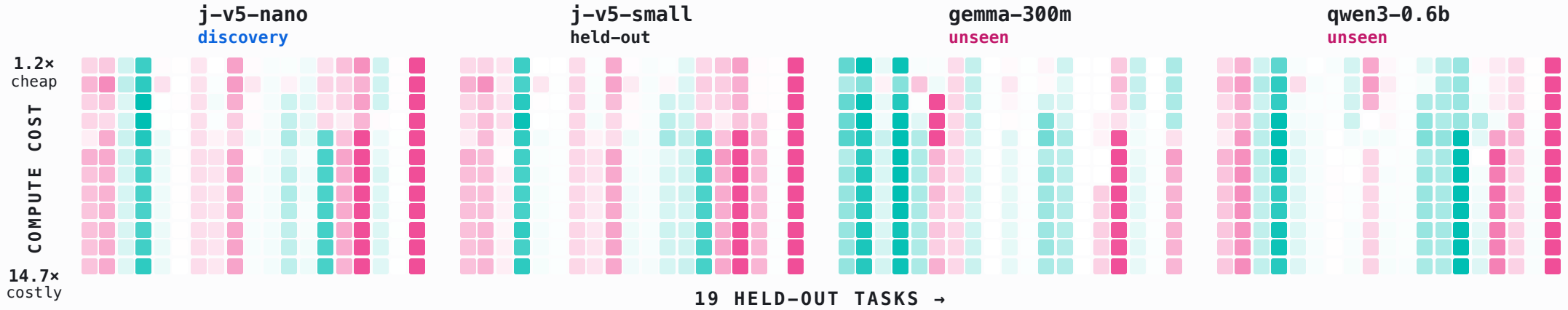
−0.98

its worst per-query cell

The transfer-selecting program at $c = 1.0$, zero extra forward passes, already beats the most expensive compute program at $c = 14.7$. Median is plotted, robust to the -0.98 tail; the pooled mean is negative too.

THE COMPUTE FRONTIER, CELL BY CELL

Compute helps about half the cells, and collapses the rest.



−0.98 +0.16

Each row is one of twelve programs ordered by compute cost (1.2× to 14.7×); each column is one of nineteen held-out tasks; four encoders, three never seen during the search. About half the cells are positive (485 of 912), and 52 of 76 task-encoder pairs improve under at least one program - yet the deep-pink cells fall to −0.98, so the pooled mean stays negative at −0.016.

THE TRANSFER-SELECTING FRONTIER

A different search finds six cheap programs that transfer.

PROGRAM	c	MEDIAN	WIN-RATE	MEAN	WORST CELL
ZeroCost-Consensus	1.00	+0.0051	68.4%	+0.0148	−0.1070
Deca-Axis	1.46	+0.0051	68.4%	+0.0133	−0.0365
Dodeca-v077	1.46	+0.0046	66.7%	+0.0126	−0.0420
Transpose-Consensus	1.50	+0.0043	83.3%	+0.0219	0.0000
Cross-Axis	1.46	+0.0025	64.9%	+0.0097	−0.0240
Penta-Axis	1.46	+0.0019	59.6%	+0.0107	−0.0485

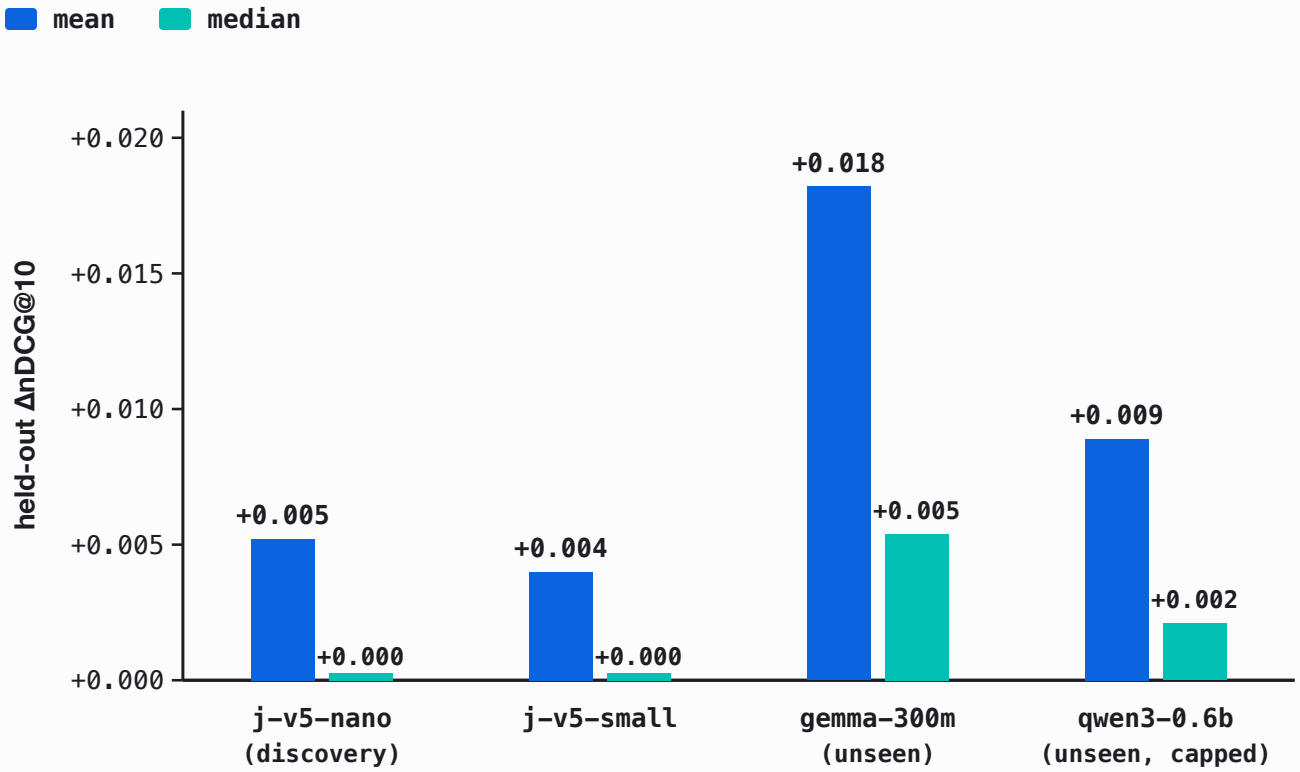
0

losing task-cells: Transpose-Consensus, at an 83% held-out win-rate

The transfer-selecting rule (the other admission objective) admits a different six - not the twelve compute programs, and all at most 1.5×. Higher cost still trends with lower held-out quality. At an 83% win-rate the best transfer program never loses a single task, and its worst per-query cell is only −0.107, almost an order of magnitude above the compute frontier's −0.98 collapse. It turns the held-out mean positive by bounding the worst case.

IT TRANSFERS

Gains are largest on encoder families never seen in discovery.

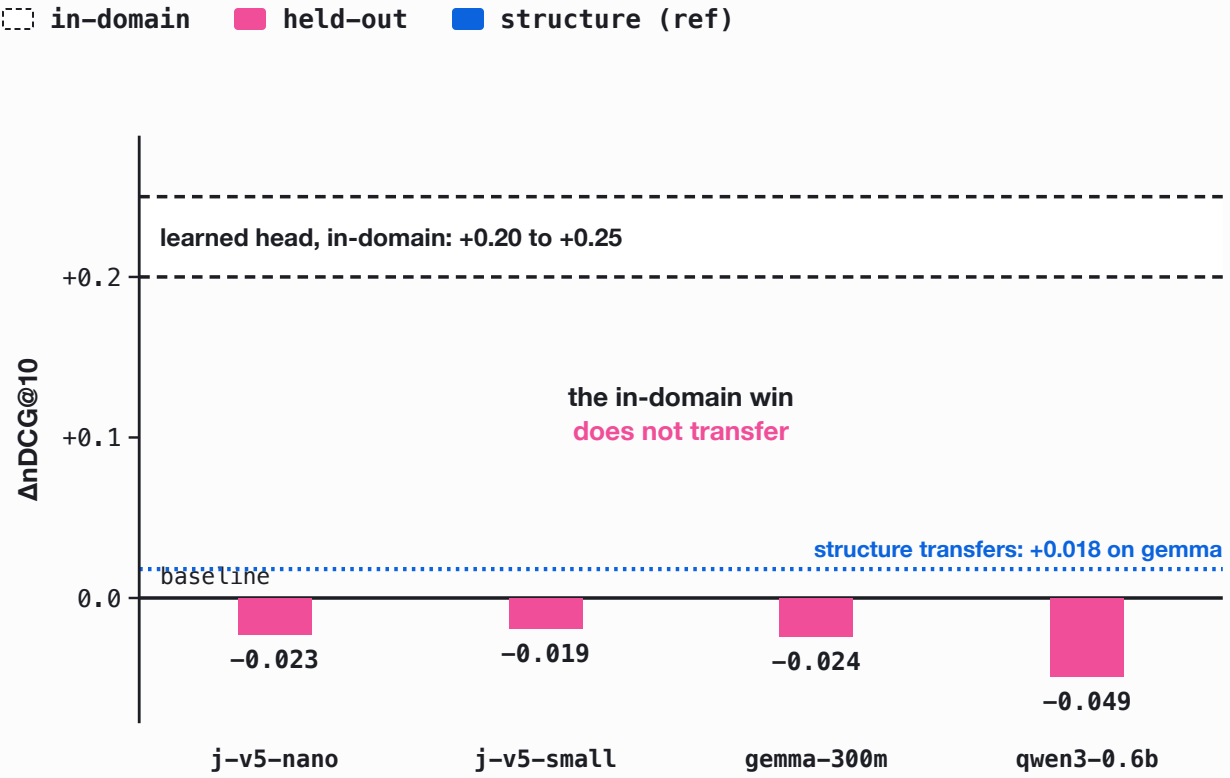


Discovered on `j-v5-nano`. Mean ΔnDCG@10 is positive on all four encoders and largest on the two families never seen during discovery; the medians sit near zero on the jina-family encoders, so the gain is concentrated in a positive tail, not broad. The transfer follows general embedding geometry, not artifacts of the discovery encoder.

> **ACROSS LANGUAGES, TOO**
>
> Applied unmodified to French and Greek: median **+0.016**, an **86%** win-rate, and every held-out cell positive on `gemma-300m`.

WHY NOT JUST TRAIN A HEAD?

A matched-budget learned head memorizes. Structure transfers.

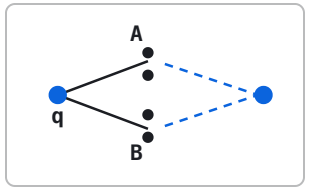


Trained on the same 14 discovery tasks, a linear, low-rank, or MLP head improves in-domain retrieval by **+0.20 to +0.25** ΔnDCG@10, yet falls below baseline on **every** held-out encoder.

Adding parameters at the same data budget does not transfer. Recombining the frozen geometry does.

WHAT THE SEARCH FOUND

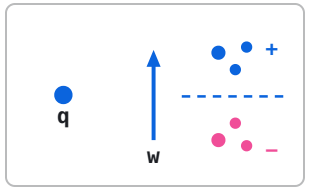
The recurring structure is classical IR, re-derived in embedding space.



Reciprocal Rank Fusion

REDISCOVERED, NOT SEEDED

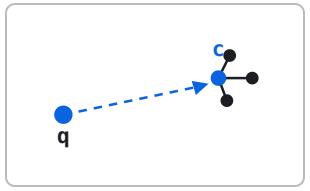
fuse two rankings into one consensus



Fisher Linear Discriminant

REDISCOVERED, NOT SEEDED

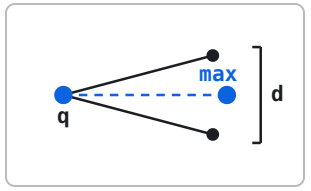
the axis that best separates relevant from not



Rocchio Pseudo-Relevance Feedback

OPERATIONALIZED FROM A SEEDED IDEA

pull the query toward the relevant centroid



Sentence-level MaxSim

OPERATIONALIZED FROM A SEEDED IDEA

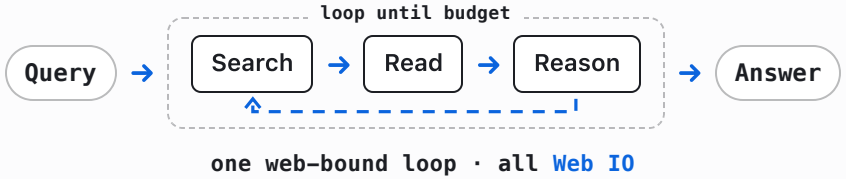
score the best sentence, not the mean

Not new programs - four classical methods the search keeps re-deriving across both frontiers: two rediscovered (RRF, Fisher), two operationalized from seeds (Rocchio, MaxSim). The structure is geometric (z-scoring, sub-document granularity, centroid feedback) and depends on cosine geometry, not model-specific training, so the cheap forms carry to encoder families never seen during discovery.

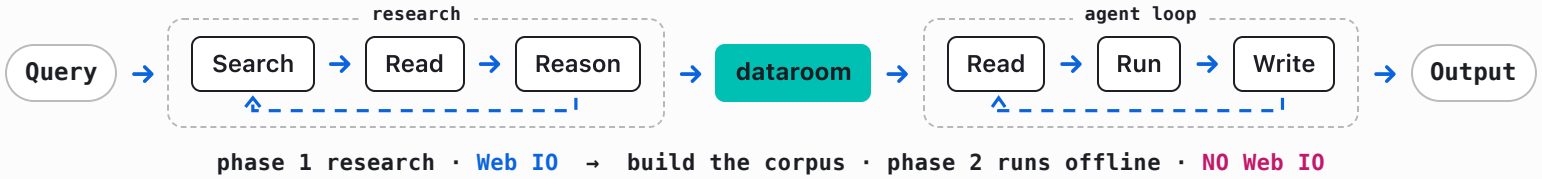
THE TREND

Test-time compute is the pattern for deep research and long-horizon tasks.

2025 · Deep research



2026 · Long-horizon tasks



Research and execution want different tokens: build the **dataroom** with cheap local ones, and save the costly frontier budget for the execution that actually needs it. That two-tier split is dataroom, then **searchbox**, next.

VERSION B · THE CORPUS STAGE

GITHUB REPO

dataroom: a loop for knowledge dump.

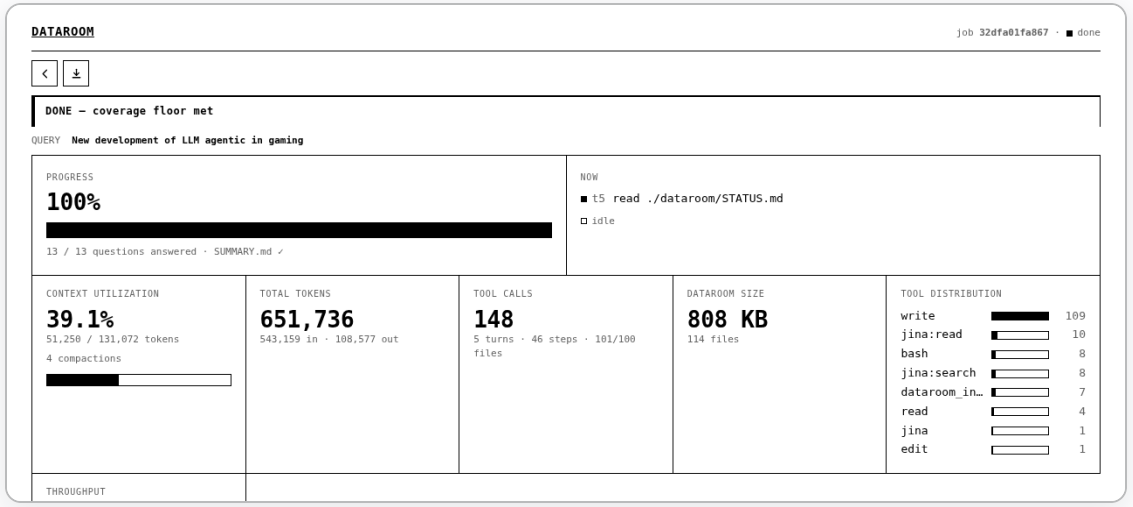
Given a token budget, spend it on **local small models** instead of a frontier model: run **search → read → write**, repeat, until the knowledge is dumped into one cited `.zip`. That dump is the dataroom - the open web **distilled** down to a small, local corpus a machine can consume.

dataroom
`build the corpus (.zip)`

↓

searchbox
`answer from the .zip`

Stage one of two: the grounded `.zip` then goes to searchbox (next) or a frontier model for the expensive second stage.



Live job page: progress to the coverage floor, token usage, and the tool-call distribution. Outcome-based stopping, not a token budget.

VERSION B · THE SEARCH STAGE

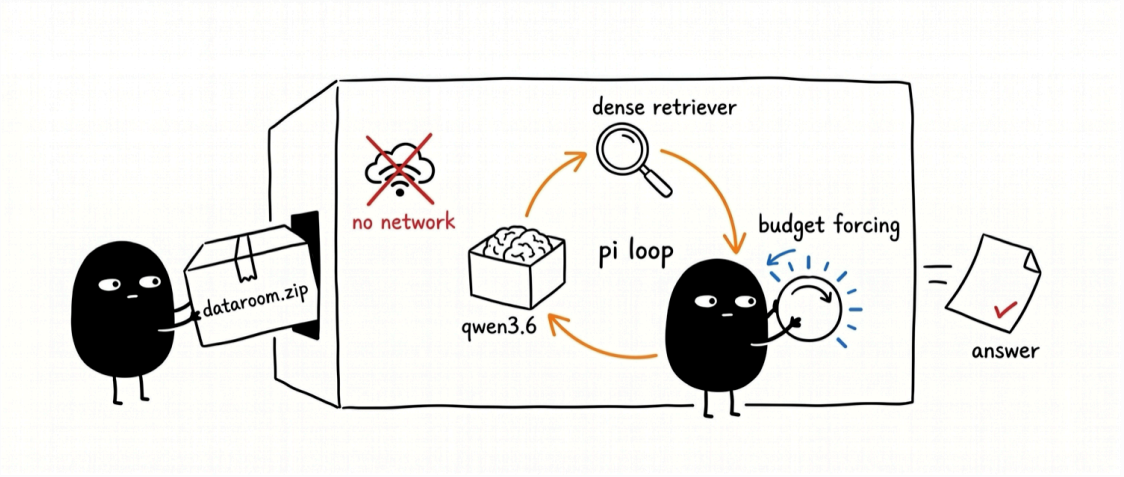
GITHUB REPO

searchbox: a testbed for studying agentic search loops.

An **airgapped** testbed for **search as test-time compute**: lock an agent in the box with one `.zip` dataroom and no web, so it can only answer by composing its own pipeline from local tools - grep, embed, rerank, similarity, cluster, select_diverse. Nothing leaks in; the search has to exhaust what is in the box.

OPEN RESEARCH QUESTIONS

- Which tool does the agent reach for first?
- Is grep all you need: where does a dense retriever add nothing?
- Does forcing more token budget (scaling TTC) help on the hard questions?



the airgapped loop, illustrated

VERSION B · THE VERIFIER

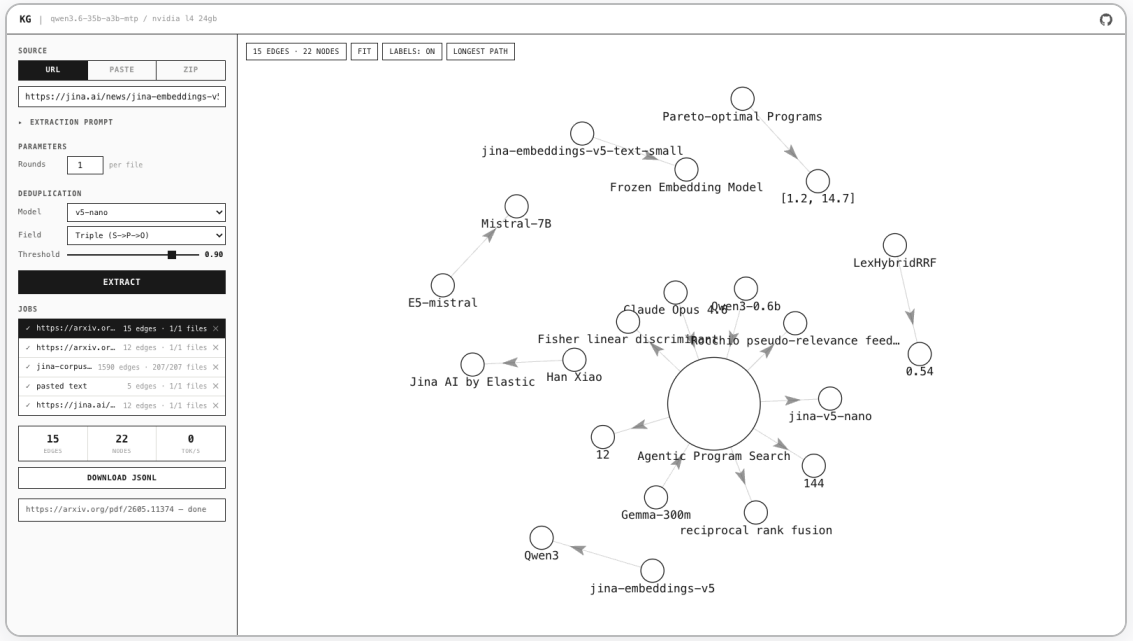
GITHUB REPO

knowledge-graph: hard multi-hop questions for a private verifier.

Trivial questions are useless for agentic search: if one grep finds the answer, every method scores the same. To **evaluate searchbox** you need questions that force the search to actually work.

HOW

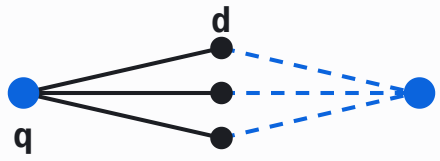
Turn the corpus into a knowledge graph - each fact a `(subject)-[predicate]->(object)` edge - then walk its **longest paths**. Those chains become **hard multi-hop questions** no single passage answers: a private, corpus-grounded eval, grown from the same corpus searchbox is locked inside.



Live graph: every fact an edge; the longest-path view surfaces the multi-hop questions.

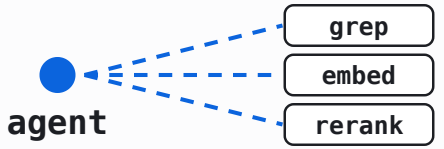
CONNECTING THE DOTS

Both manufacture a search pipeline at test time. Neither grows the model.



A `Autoresearch programs`

PIPELINE	multi-pass embedding algebra
WHAT SCALES	structure, not forward passes



B `searchbox / agentic search`

PIPELINE	a chain of retrieval tools
WHAT SCALES	tool composition, not parameters

AI Engineer World's Fair

THANK YOU

Information retrieval is test-time compute.

Han Xiao · VP of AI, Elastic

github.com/hanxiao · arXiv:2605.11374

@hxiao · in/hxiao87

FOLLOW ON X

x.com/hxiao

THESE SLIDES

hanxiao.io/aie-sf-2026